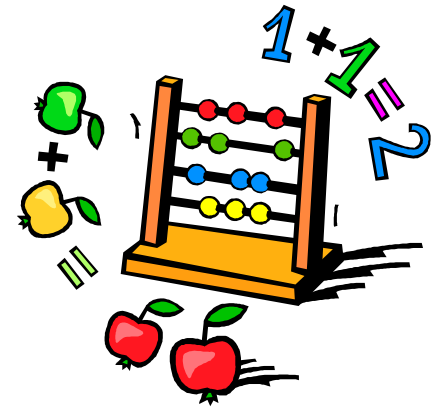


sets



Java Interfaces

The following are important interfaces included in the Java language ::

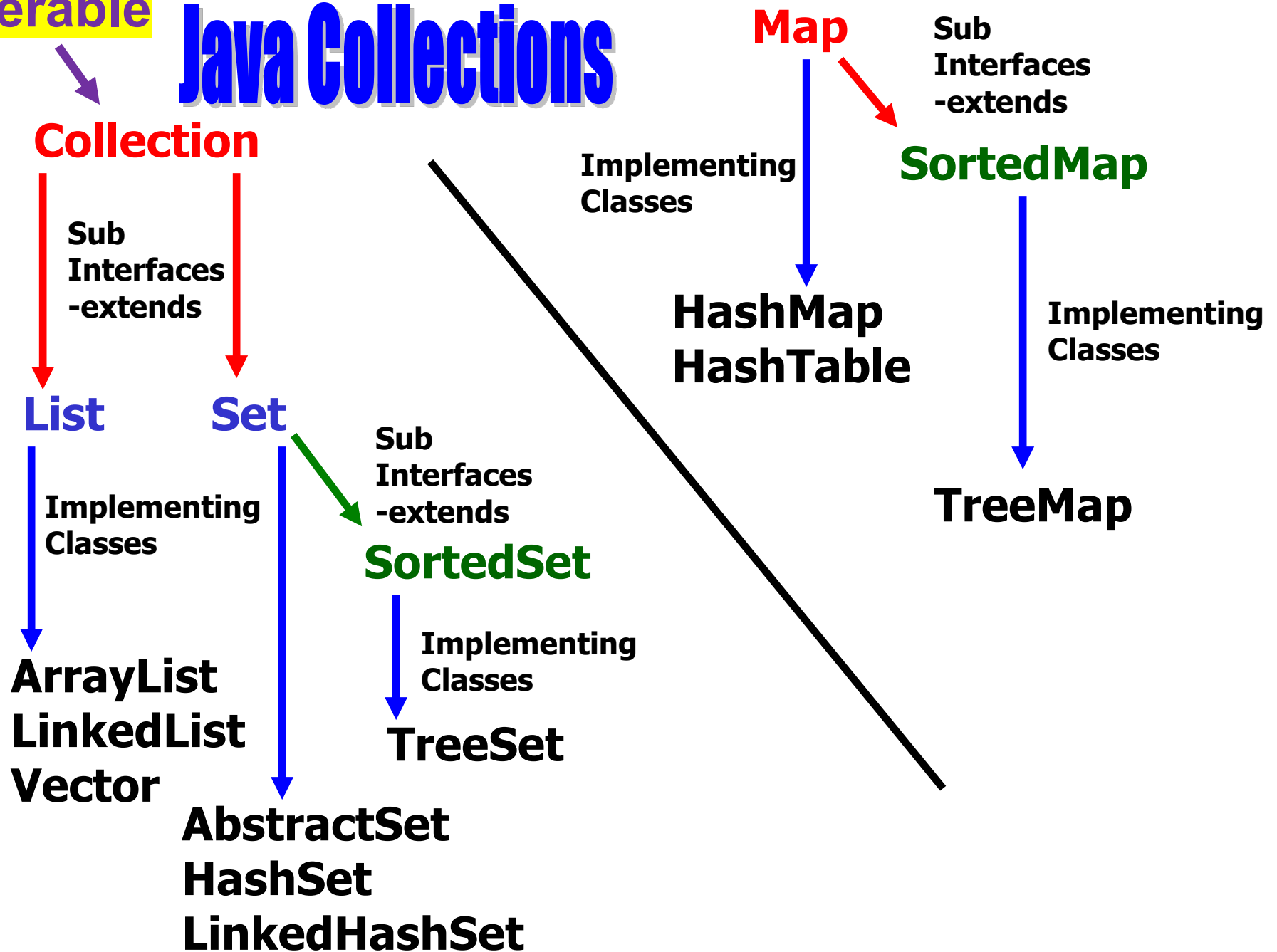
Collection

Set

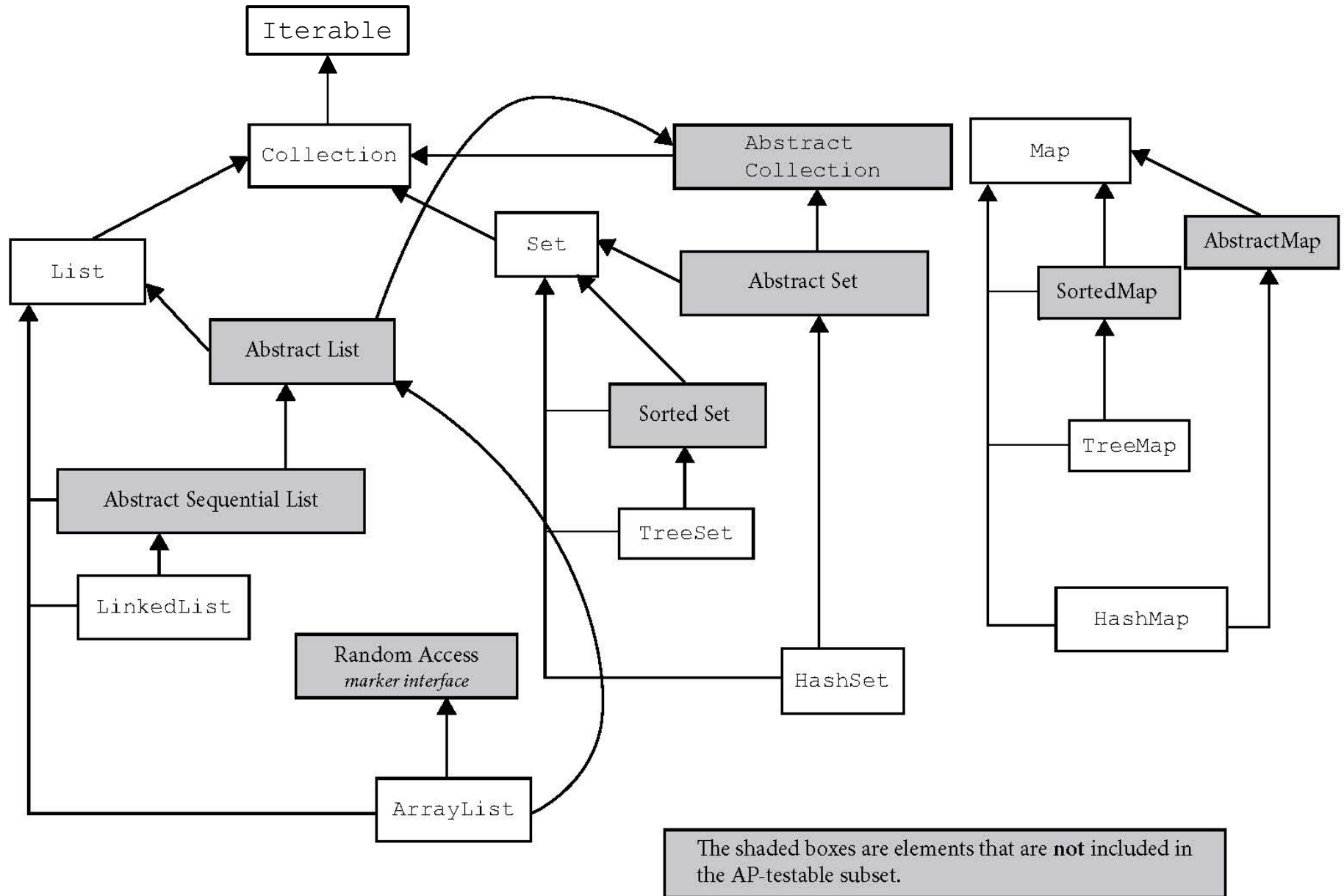
Map

Iterable

Java Collections

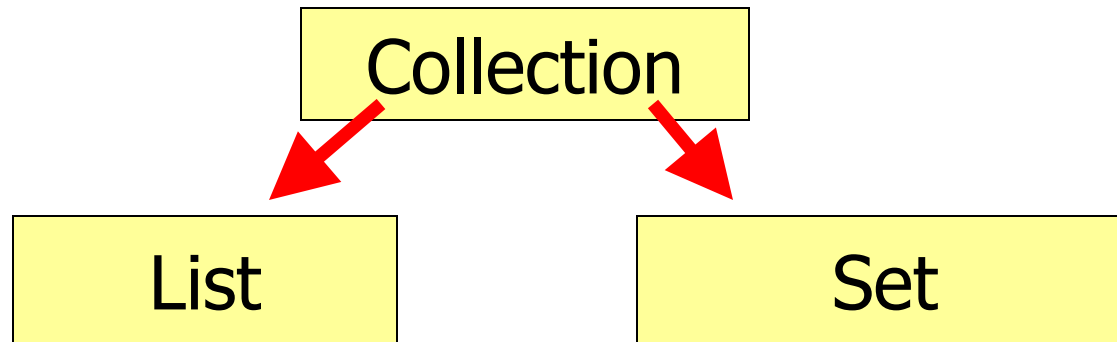


The Java Collections Hierarchy



The Collection Interface

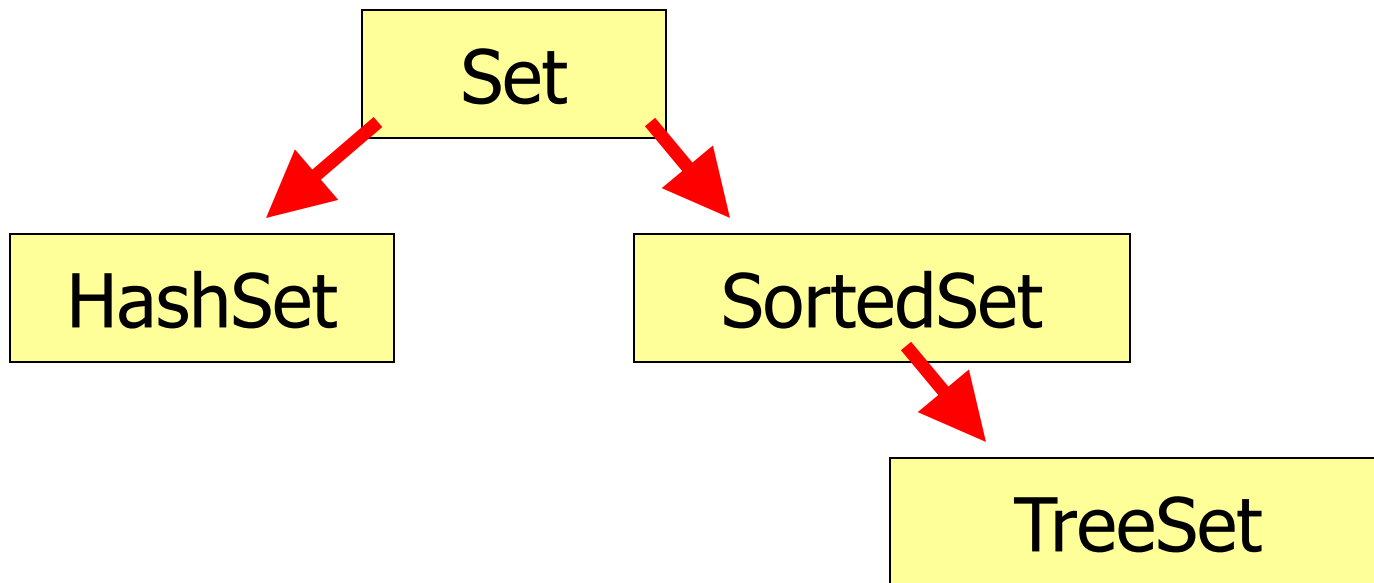
The Collection interface is the parent of List and Set. The Collection interface has many methods listed including add(), clear(), remove(), and size().



others not shown

The Set Interface

The Set interface extends the Collection interface.



set

A set is a group of items all of the same type of which none are duplicates.

Set

Because Set is an interface, you cannot instantiate it.

Set bad = new Set(); **//illegal**

Set set = new HashSet(); **//legal**
set = new TreeSet(); **//legal**

hash and tree store Object references.

Set

You should always specify which type of reference you want to store in the TreeSet or HashSet.

```
Set<Double> bytes = new TreeSet<Double>();
```

```
Set<String> its = new HashSet<>();
```

set

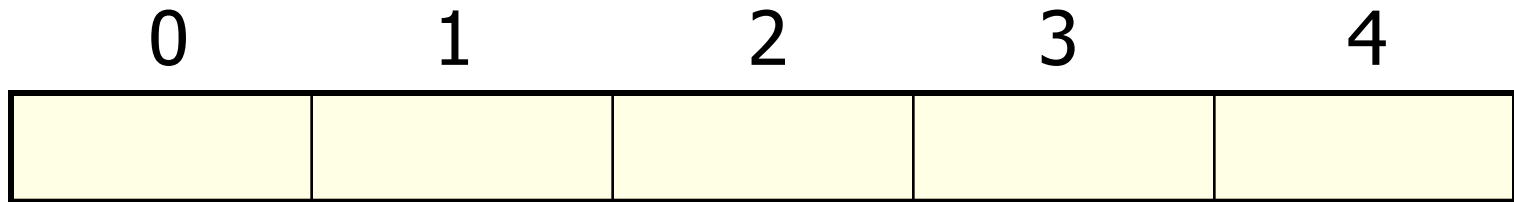
HashSet – a set ordered by each item's hashCode that is extremely time efficient.

TreeSet – a naturally ordered set that is very efficient, but not as efficient as HashSet.

HashTable

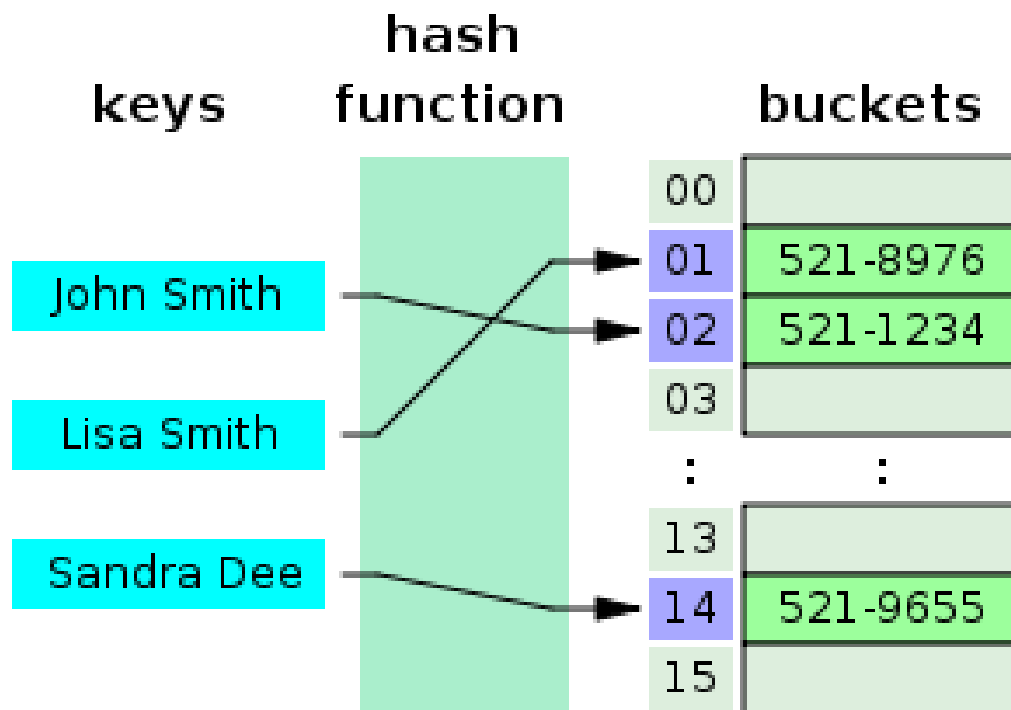
HashSet and HashMap were both created around hash tables.

A hash table is essentially a giant array. Each item is inserted into the array according to a hash formula.



HashTable

The hash function calculates an index from the data item's key and use this index to place the data into the array.



HashTable

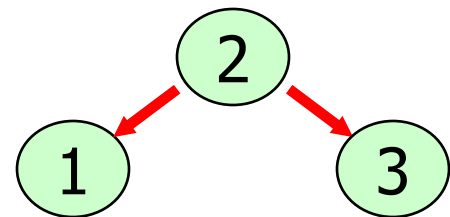
The advantage of hash tables is their speed.

The disadvantage is that you don't know what order you'll iterate over the contents.

Binary Tree

TreeSet and TreeMap were built around binary trees.

A Binary Tree is a group of nodes that contain left and right references. Each item is inserted into the tree according to its relationship to the other nodes.



Set Methods

Set

frequently used methods

	Name	Use
boolean	add(obj)	adds item obj to the set
boolean	remove(obj)	removes item obj from the set
void	clear()	removes all items from the set
int	size()	returns the # of items in the set

Why does the Set interface not include corollaries to these List methods?

get(int index)

remove(int index)

add(int index, E element)

set(int index, E element)

HashSet add

```
Set<Integer> intSet;  
intSet = new HashSet<Integer>();  
intSet.add(45);  
System.out.println(intSet.add(12));  
System.out.println(intSet.add(12));  
intSet.add(23);  
System.out.println(intSet);
```

OUTPUT

true

false

[45, 23, 12]

HashSet add

```
Set<String> stringSet;  
stringSet = new HashSet<String>();  
stringSet.add("AB");  
stringSet.add("23");  
stringSet.add("ab");  
System.out.println(stringSet);
```

OUTPUT
[ab, 23, AB]

Open

hashsetint.java

hashsetstring.java

Quick Creation of a Set

```
String[] words = "I am Crazy I am".split(" ");  
List<String> wordList = Arrays.asList(words);  
Set<String> set = new TreeSet<String>(wordList);  
System.out.println(set);
```

OUTPUT

[Crazy, I, am]

**Open
SetSplit.java**

HashSet removed

```
Set<Double> doubleSet;  
doubleSet = new HashSet<Double>();  
doubleSet.add(2.5);  
doubleSet.add(5.8);  
doubleSet.add(7.3);  
System.out.println(doubleSet);  
doubleSet.remove(5.8);  
System.out.println(doubleSet.remove(0));  
System.out.println(doubleSet);
```

OUTPUT

[7.3, 2.5, 5.8]

false

[7.3, 2.5]

Open

hashsetremove.java

TreeSet add

```
Set<Integer> intSet;  
intSet = new TreeSet<Integer>();  
System.out.println(intSet.add(45));  
intSet.add(12);  
System.out.println(intSet.add(12));  
intSet.add(23);  
System.out.println(intSet);
```

TreeSets behave exactly
like HashSets, except —→
they are in order:

OUTPUT

true

false

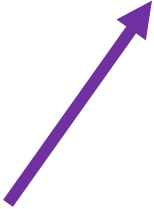
[12, 23, 45]

TreeSet add

```
Set<String> stringSet;  
stringSet = new TreeSet<String>();  
stringSet.add("AB");  
stringSet.add("23");  
stringSet.add("ab");  
System.out.println(stringSet);
```

OUTPUT
[23, AB, ab]

In order!



Open

treesetint.java

treesetstring.java

TreeSet removed

```
Set<Double> doubleSet;  
doubleSet = new TreeSet<Double>();  
doubleSet.add(2.5);  
doubleSet.add(5.8);  
doubleSet.add(7.3);  
System.out.println(doubleSet);  
doubleSet.remove(5.8);  
doubleSet.remove(0.0);  
System.out.println(doubleSet);
```

OUTPUT

[2.5, 5.8, 7.3]
[2.5, 7.3]



In order!

Open

treeSetremove.java

Sets aren't Lists!

List methods Sets don't have:

get(i)

indexOf(E)

listIterator()

remove(i)

set(i, E)

Open

setoutputput.java

Iterating over Sets

Sets aren't Lists!

This means you can't iterate over a Set via the indices (since there are none!):

```
for (int i=0; i<set.size(); i++)  
    out.print(set.get(i) + "");
```

Looping thru a Set Using a for-each loop

```
Set<Double> set = new TreeSet<Double>();  
set.add(5.8);  
set.add(7.3);  
set.add(2.5);
```

```
for(double num : set)  
{  
    System.out.println(num);  
}
```

OUTPUT

2.5

5.8

7.3

Looping thru a Set Using an iterator

```
Set<Double> set = new TreeSet<>();  
set.add(5.8);  
set.add(7.3);  
set.add(2.5);
```

```
Iterator<Double> it = set.iterator();  
while(it.hasNext())  
{  
    double num = it.next();  
    out.println(num);  
}
```

OUTPUT

2.5

5.8

7.3

Using the Collection forEach Method and Lambda Expressions

```
Set<Double> set = new TreeSet<>();  
set.add(5.8);  
set.add(7.3);  
set.add(2.5);  
  
set.forEach(  
    num -> out.println(num)  
);
```

OUTPUT

2.5

5.8

7.3

Uses the Set's iterator, so same order as the normal for-each loop.

Open

setoutputnew.java

setsplit.java

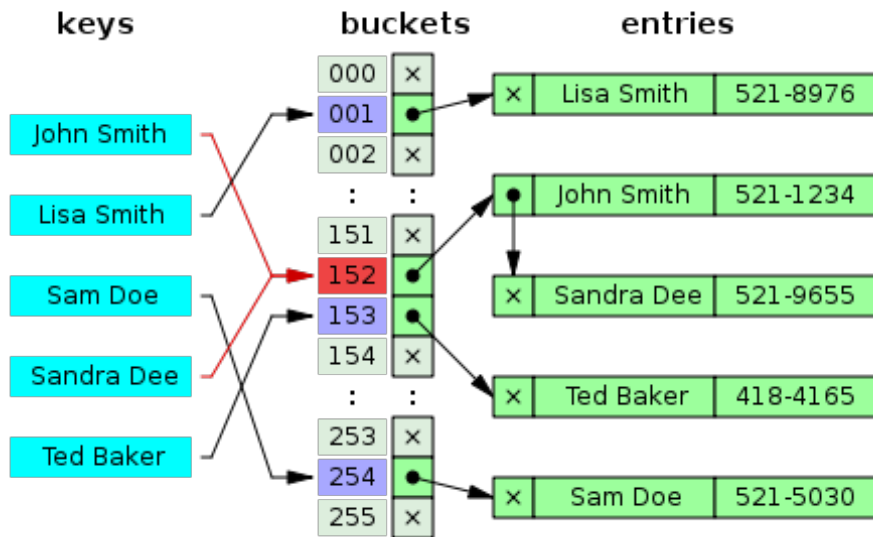
Big O

Big-O

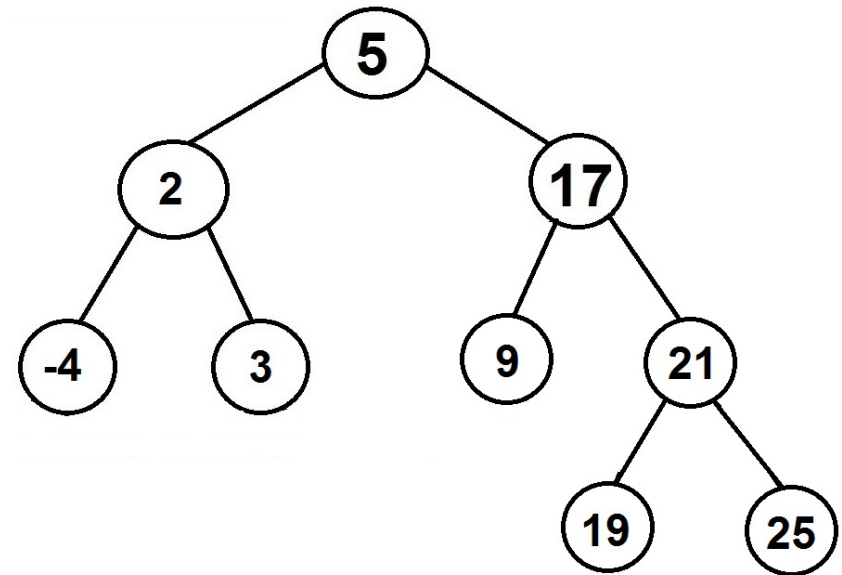
frequently used notations

Name	Notation
constant	$O(1)$
logarithmic	$O(\log_2 N)$
linear	$O(N)$
linearithmic	$O(N \log_2 N)$
quadratic	$O(N^2)$
exponential	$O(2^n)$

Hash Table



Binary Search Tree



Set Big O

	Tree Set	Hash Set
add	$O(\log_2 N)$	$O(1)$
remove	$O(\log_2 N)$	$O(1)$
contains	$O(\log_2 N)$	$O(1)$
size	$O(1)$	$O(1)$
iterate over set	$O(n)$	$O(n)$

TreeSets are implemented with balanced binary trees (red/black trees).

HashSets are implemented with hash tables.

**Now it is time for
another round of**

What is the Big-O?

What is the Big-O?

```
int n = list.size();  
Set s = new HashSet();  
for(int i=0; i<n; i++)  
    s.add(list.get(i));
```

Big O Notation – $O(N)$

The work needed to add each element of ray to s would be $N \times 1$. Ray has N items and add() for HashSet has an $O(1)$ bigO.

What is the Big-O?

```
int n = list.size();  
Set s = new TreeSet();  
for(int i=0; i<n; i++)  
    s.add(list.get(i));
```

Big O Notation – $N * \log_2(N)$

The work needed to add each element of ray to s would be $N * \log_2 N$. Ray has N items and add() for TreeSet has a $\log_2$ bigO.

Set Theory

$$A = \{0, 1, 2, 3, 4\} \quad B = \{0, 2, 4, 6, 8\}$$

$$\text{Union: } A \cup B = \{0, 1, 2, 3, 4, 6, 8\}$$

$$\text{Intersection: } A \cap B = \{0, 2, 4\}$$

$$A - B: A - B = \{1, 3\}$$

$$B - A: B - A = \{6, 8\}$$

Symmetric

$$\text{difference: } A \Delta B = \{1, 3, 6, 8\}$$

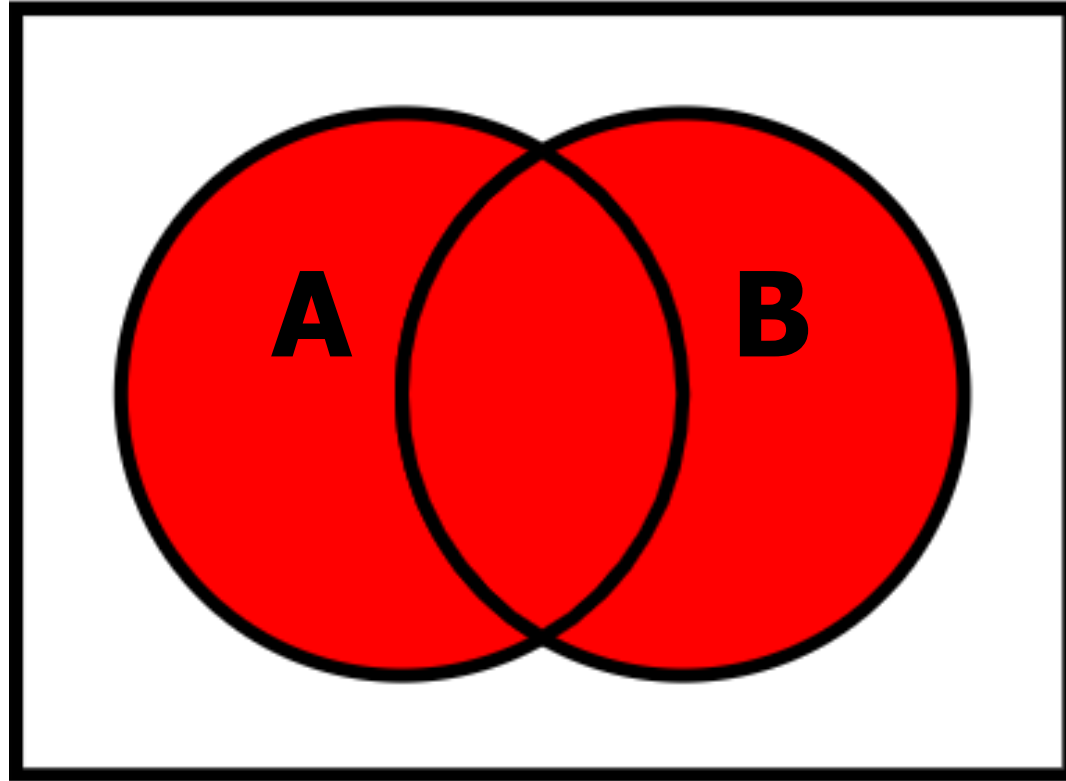
Set methods

addAll

removeAll

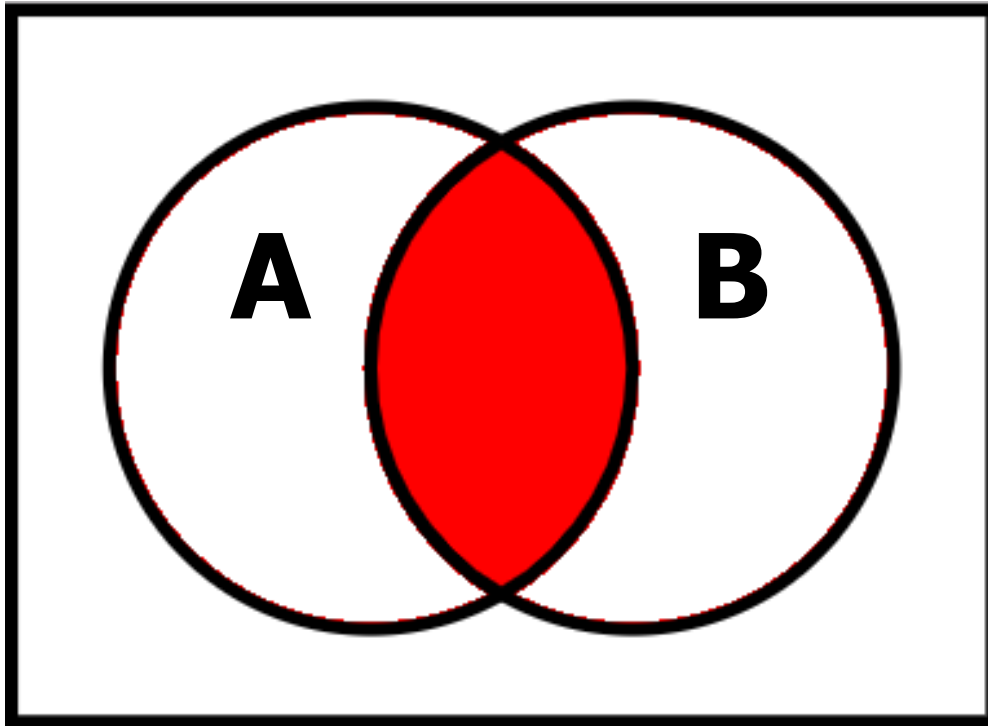
retainAll

Union



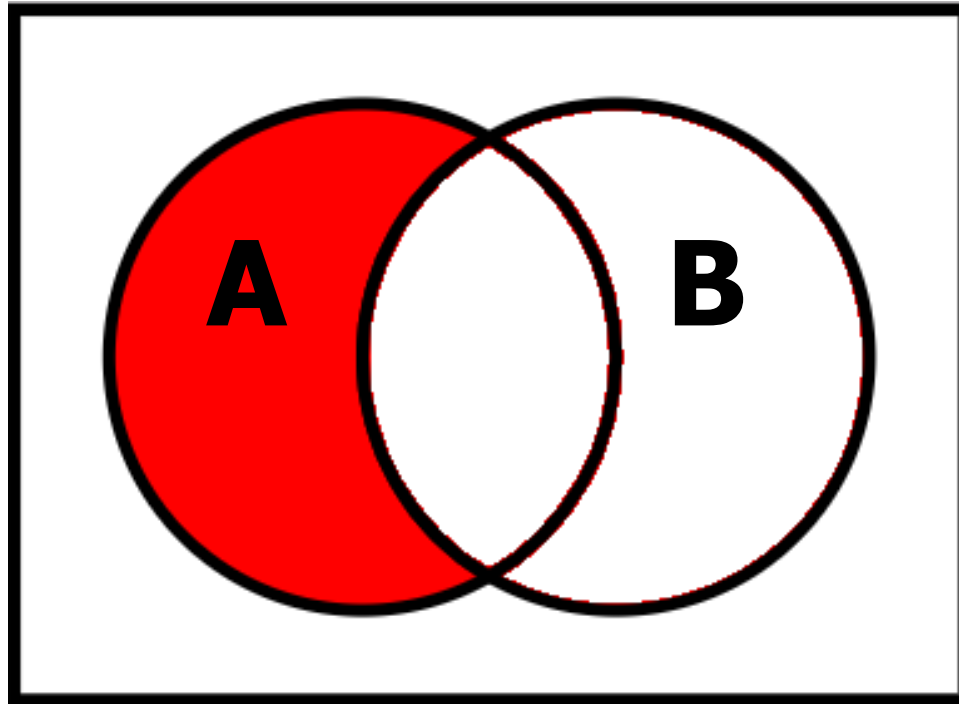
```
union.addAll(A)  
union.addAll(B)
```


Intersection



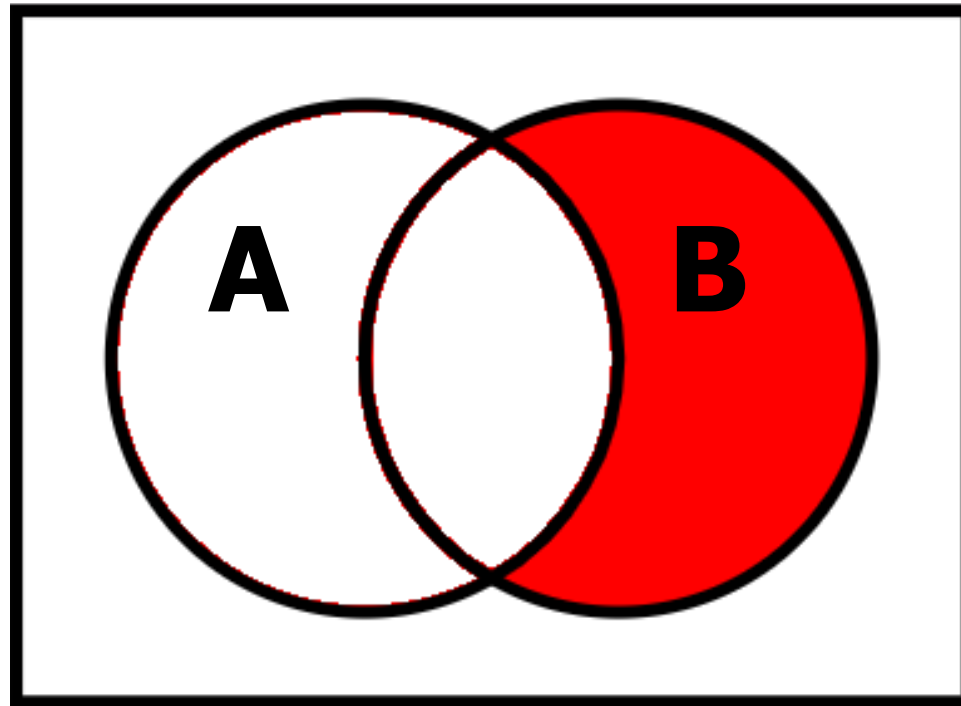
intersection.addAll(A)
intersection.retainAll(B)

A minus B



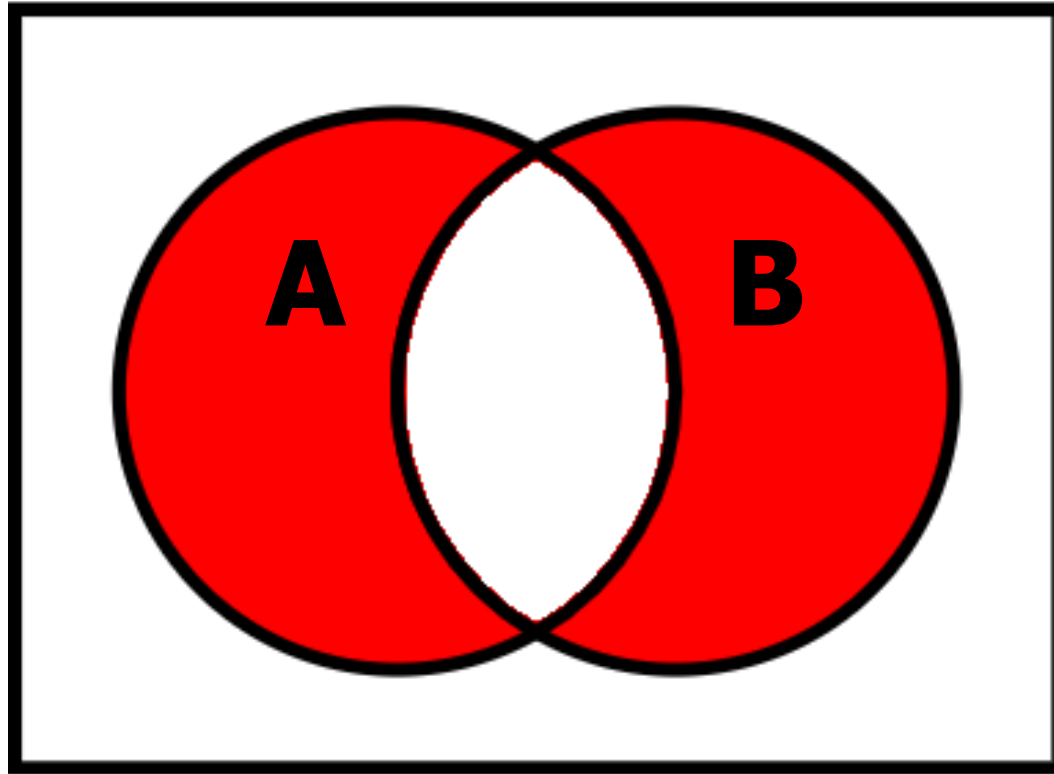
diff.addAll(A)
diff.removeAll(B)

B minus A



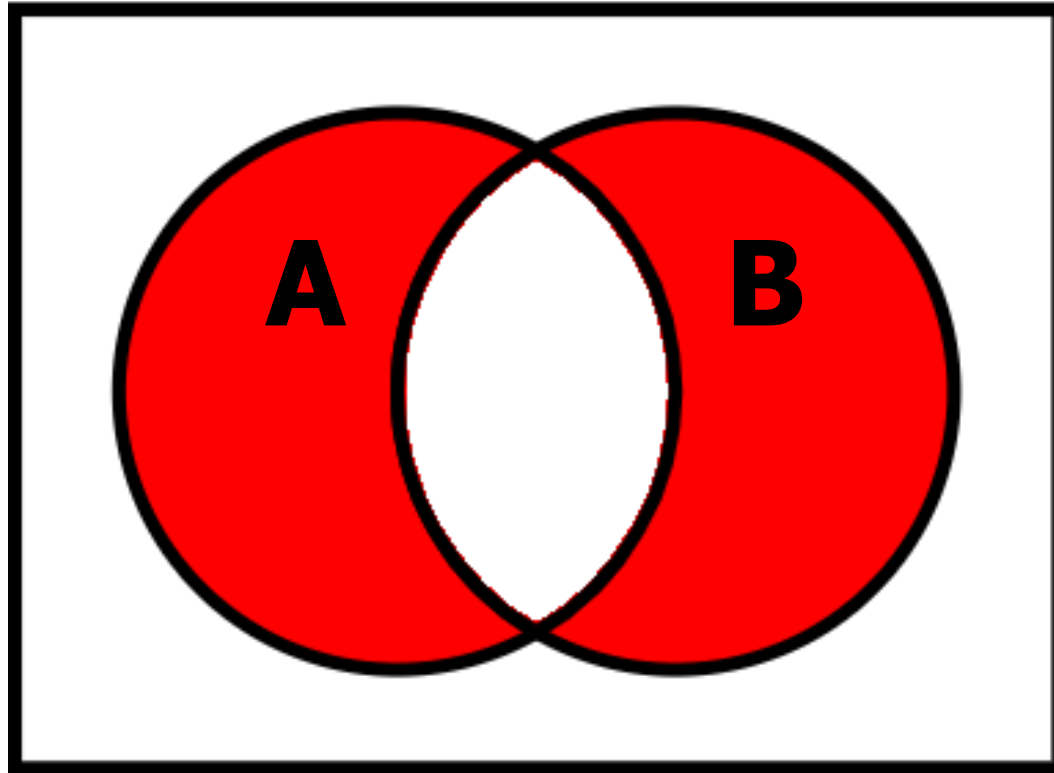
diff.addAll(B)
diff.removeAll(A)

Symmetric Difference



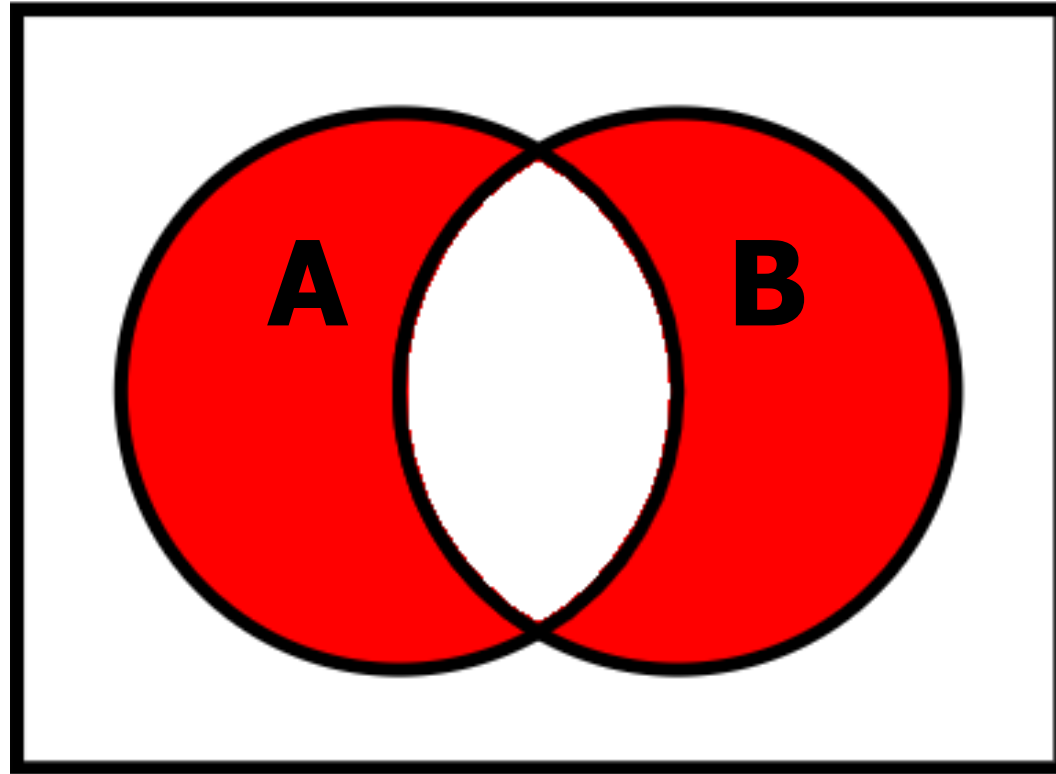
Two ways!

Symmetric Difference



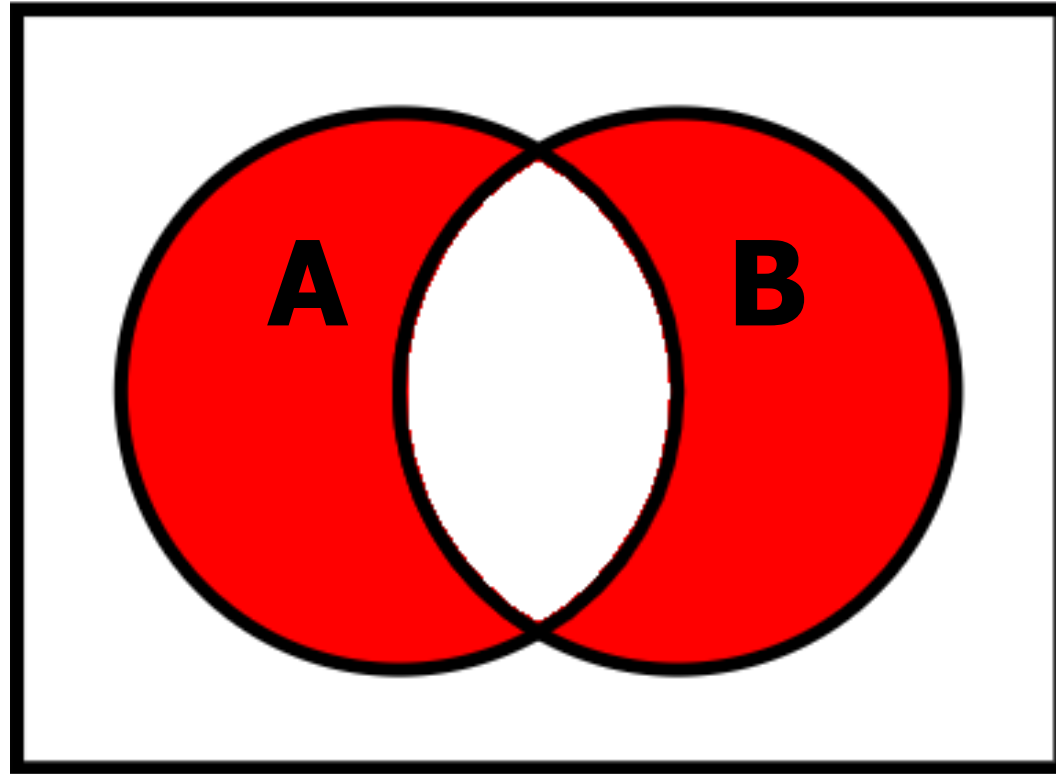
```
diff = new set  
diff.addAll(aMinusB(A, B))  
diff.addAll(bMinusA(A, B))
```

Symmetric Difference



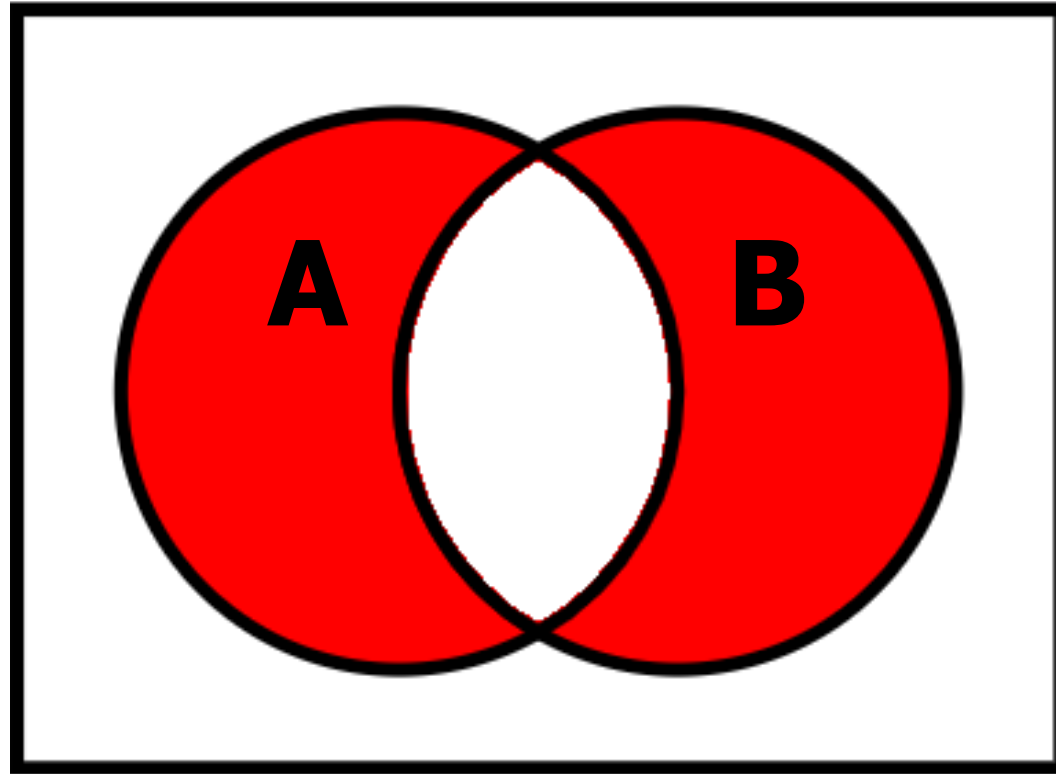
diff = **union**(**aMinusB**(A, B), **bMinusA**(A, B))

Symmetric Difference



diff = union(A, B)
diff.remove(intersection(A, B))

Symmetric Difference



diff = **aMinusB**(**union**(A, B), **intersection**(A, B))

Using the Stream forEach Method

Parallel streams allow us to execute the stream in multiple threads, and in such situations, the execution order is undefined. Java only requires all threads to finish before any terminal operation, such as *Collectors.toList()*, is called.

Using the Stream forEach Method

Calling **add** or **remove** while iterating using the Stream forEach will lead to a `ConcurrentModificationException` being thrown:

```
Consumer<String> removeElement = s -> {  
    System.out.println(s + " " + list.size());  
    if (s != null && s.equals("A")) {  
        list.remove("A");  
    }  
};
```

```
list.stream().forEach(removeElement);
```

Just like it would if you used the Collection `forEach` method:

```
list.forEach(removeElement);
```

Using the Stream forEach Method

If you're using the Collection `forEach` method, it's okay to modify elements while iterating, for instance by calling the list **`set`** method.

However, Java requires an operation on a stream to be non-interfering. This means that elements shouldn't be modified during the execution of the stream pipeline.

Using the Stream forEach Method

```
Set<Double> set = new TreeSet<>();  
set.add(5.8);  
set.add(7.3);  
set.add(2.5);  
  
set.stream().forEach(  
    num -> out.println(num)  
);
```

OUTPUT

2.5
5.8
7.3

Creates a Stream. Calls its forEach method. Doesn't follow encounter order. Better for parallelism.

Using the Stream `forEachOrdered` Method

```
Set<Double> set = new TreeSet<>();  
set.add(5.8);  
set.add(7.3);  
set.add(2.5);  
  
set.stream().forEachOrdered(  
    num -> out.println(num)  
);
```

OUTPUT

2.5
5.8
7.3

**Follows the encounter order of the Stream,
which is natural order in this case.**